

FUNDAMENTOS DA LÓGICA DE PROGRAMAÇÃO COM PORTUGOL

Um Guia Prático para Iniciantes

Instrutor: Djalma Batista Barbosa Junior



Capítulo 1:

A Base do Pensamento Computacional

1.1 Desvendando a Lógica de Programação: O Que é e Por Que é Essencial

No coração de cada aplicativo, site ou jogo que utilizamos, reside um conceito fundamental: a lógica de programação. Longe de ser um jargão técnico inacessível, a lógica de programação é, em sua essência, a habilidade de organizar o pensamento de forma estruturada e sequencial para resolver um problema.

É o conjunto de regras e técnicas que permite a uma pessoa desenvolvedora decompor um desafio complexo em etapas menores e mais simples, criando um roteiro claro que um computador possa seguir.¹

A importância dessa habilidade não pode ser subestimada, pois ela transcende qualquer linguagem de programação específica. Aprender a programar em Python, Java ou C# é como aprender a falar diferentes idiomas; a lógica de programação, por sua vez, é a gramática universal que governa todos eles. Uma vez que os fundamentos lógicos são dominados, aprender uma nova linguagem de programação torna-se uma tarefa muito mais simples, pois a estrutura de pensamento já está consolidada.¹

Dominar a lógica de programação traz benefícios diretos e duradouros para qualquer aspirante a desenvolvedor. Primeiramente, ela cultiva uma abordagem sistemática para a resolução de problemas, ensinando a analisar desafios, identificar padrões e desenvolver soluções passo a passo.

Em segundo lugar, um código construído sobre uma lógica sólida é

inerentemente mais eficiente, consumindo menos recursos computacionais e executando tarefas de forma mais rápida. Por fim, programas com uma lógica clara são mais fáceis de ler, manter e atualizar no futuro, uma qualidade indispensável no desenvolvimento de software profissional.

Em resumo, a lógica de programação não é apenas o primeiro passo, mas a base contínua sobre a qual toda a carreira de um desenvolvedor é construída

1.2 Algoritmos:

A Receita para Resolver Problemas

Se a lógica de programação é a maneira de pensar, o algoritmo é o resultado tangível desse pensamento: um plano de ação detalhado. Um algoritmo é definido como uma sequência finita e ordenada de instruções claras e não ambíguas, projetada para realizar uma tarefa específica ou resolver um problema. A analogia mais poderosa e universal para entender um algoritmo é a de uma receita de bolo.

A receita lista os ingredientes necessários (os dados de entrada), descreve um conjunto de passos sequenciais para misturá-los e assá-los (o processamento) e, ao final, resulta em um bolo pronto (o resultado ou saída).

Essa estrutura de entrada, processamento e saída é a espinha dorsal de todo algoritmo. Os algoritmos estão presentes em nosso cotidiano, muitas vezes de forma implícita. A rotina de se preparar para o trabalho pela manhã, as instruções para montar um móvel ou o caminho que um aplicativo de mapas traça são todos exemplos de algoritmos.

Eles pegam um estado inicial (não estar pronto para o trabalho, peças desmontadas, localização atual) e, através de uma série de passos bem definidos, chegam a um estado final desejado (pronto para o trabalho, móvel montado, chegada

ao destino).

No contexto da computação, essa ideia é formalizada. Um algoritmo simples para somar dois números, por exemplo, pode ser descrito da seguinte forma:

1. **Início**
2. **Entrada:** Receber o primeiro número.
3. **Entrada:** Receber o segundo número.
4. **Processamento:** Somar os dois números recebidos e guardar o resultado.
5. **Saída:** Mostrar o resultado da soma.
6. **Fim**

Este exemplo simples ilustra as características essenciais de um bom algoritmo: cada passo é claro, a sequência é lógica e o processo tem um início e um fim definidos. É essa "receita" que será posteriormente traduzida para uma linguagem que o computador entende.

1.3 De Ideias a Instruções:

Pseudocódigo e a Linguagem Portugol

Uma vez que a solução para um problema foi estruturada como um algoritmo, o próximo passo é descrevê-la de uma forma que se aproxime da programação, mas sem as complexidades e a rigidez da sintaxe de uma linguagem real. É aqui que entra o pseudocódigo. O pseudocódigo, ou português estruturado, é uma ferramenta para representar algoritmos usando uma linguagem natural e simplificada, mas que já incorpora convenções estruturais da programação, como palavras-chave para início, fim, leitura de dados e decisões.¹² Ele serve como uma ponte crucial entre o pensamento humano e o código de máquina.

Dentro do universo do pseudocódigo, o Portugol se destaca como uma

pseudolinguagem formalizada, criada especificamente com o propósito de ensinar lógica de programação para falantes da língua portuguesa.

Desenvolvido no Brasil, o Portugol utiliza comandos em português (leia, escreva, se, entao, para faca), o que remove uma barreira significativa para iniciantes: a necessidade de aprender palavras-chave em inglês ao mesmo tempo em que se esforçam para compreender os conceitos lógicos.

Muitos estudantes podem se perguntar: "Por que não começar diretamente com uma linguagem de mercado como Python ou JavaScript?". A resposta reside na separação de preocupações.

Aprender uma linguagem de programação real envolve dominar não apenas a lógica, mas também uma sintaxe rigorosa, regras de pontuação, gerenciamento de memória (em algumas linguagens como C) e um vasto ecossistema de bibliotecas.¹

Essa carga cognitiva pode desviar o foco do objetivo principal, que é aprender a pensar de forma algorítmica.

O Portugol funciona como um "ambiente de treinamento controlado", uma academia para o raciocínio lógico. Ele permite que o estudante se concentre exclusivamente em como estruturar variáveis, criar condições e construir laços de repetição.

Ao isolar o aprendizado da lógica, o Portugol capacita o aluno a construir uma base mental sólida. Uma vez que essa base esteja firme, a transição para qualquer linguagem de programação se torna uma questão de aprender uma nova sintaxe para expressar os mesmos conceitos lógicos já dominados.

Capítulo 2:

Primeiros Passos no Mundo do Código

2.1 Ambiente de Desenvolvimento:

Introdução ao Portugol Studio

Para transformar os algoritmos em programas executáveis, é necessário um ambiente de desenvolvimento — uma ferramenta que permita escrever, testar e depurar o código. No universo do Portugol, duas ferramentas são historicamente populares: o VisuAlg e o Portugol Studio.

Embora o VisuAlg tenha sido fundamental para o ensino de programação por muitos anos, o **Portugol Studio** é a ferramenta recomendada para os iniciantes de hoje. Ele possui uma interface mais moderna e intuitiva, é mantido ativamente pela comunidade acadêmica e oferece recursos mais avançados que facilitam o aprendizado, como mensagens de erro mais claras e depuração passo a passo.¹⁸

O Portugol Studio pode ser baixado gratuitamente e instalado em sistemas Windows, Linux e macOS. Existe também uma versão web, o Portugol Web Studio, que permite programar diretamente no navegador, sem a necessidade de instalação.

Ao abrir o Portugol Studio, a interface principal apresenta três áreas essenciais:

1. **Editor de Código:** A área central, onde o pseudocódigo é escrito.
2. **Console:** A área na parte inferior, onde os resultados do programa (saídas) são exibidos e onde o usuário pode inserir dados (entradas).
3. **Barra de Ferramentas:** Na parte superior, com botões importantes como "Executar" (para rodar o programa) e "Depurar" (para executar o código linha a linha, uma ferramenta poderosa para encontrar erros).

Familiarizar-se com este ambiente é o primeiro passo prático para começar a

dar vida às ideias lógicas.

2.2 Seu Primeiro Programa:

"Olá, Mundo!" com escreva ()

Todo programa em Portugol segue uma estrutura básica e padronizada, que serve como um esqueleto para o nosso código. Essa estrutura é composta por três partes principais:

- algoritmo "nome_do_algoritmo": A primeira linha, que define o início do programa e dá um nome a ele. O nome deve estar entre aspas duplas.
- início: Palavra-chave que marca o começo do bloco de comandos que serão executados.
- finalgoritmo: Palavra-chave que marca o fim do programa.

A tradição em programação dita que o primeiro programa a ser escrito é o "Olá, Mundo!", um programa simples que apenas exibe essa mensagem na tela. Em Portugol, isso é feito com o comando escreva ().

Snippet de código

```
algoritmo "OlaMundo"
```

```
// Este é um comentário. O programa ignora tudo que vem depois de duas barras.
```

```
// O objetivo deste programa é apenas exibir uma mensagem de boas-vindas.
```

```
início
```

```
    escreva("Olá, Mundo!")
```

```
finalgoritmo
```

Ao executar este código no Portugol Studio, o texto "Olá, Mundo!" aparecerá no console. O comando escreva() instrui o computador a exibir na tela o que quer que esteja dentro dos parênteses. Se for um texto, ele deve estar entre aspas duplas.

Uma variação importante deste comando é o `escreval()`. A diferença é sutil, mas fundamental:

- `escreva()`: Exibe o conteúdo e mantém o cursor na mesma linha.
- `escreval()`: Exibe o conteúdo e move o cursor para a linha seguinte, como se a tecla "Enter" fosse pressionada.

Observe a diferença na prática:

Snippet de código

```
algoritmo "DiferencaEscreva"  
inicio  
    escreva("Isso fica na primeira linha. ")  
    escreva("Isso também.")  
    escreval("") // Pula uma linha  
    escreval("Isso fica na segunda linha.")  
    escreval("E isso na terceira.")  
fimalgoritmo
```

Saída no Console:

Isso fica na primeira linha. Isso também.

Isso fica na segunda linha.

E isso na terceira.

Dominar esses dois comandos de saída é o primeiro passo para que seus programas possam se comunicar com o usuário.

2.3 Armazenando Dados:

Variáveis e Tipos Primitivos

Programas de computador raramente trabalham com dados fixos; eles precisam armazenar e manipular informações que podem mudar durante a execução, como a idade de um usuário ou o preço de um produto. Para isso, utilizamos **variáveis**. Uma variável pode ser imaginada como uma "caixa" ou um "contêiner" na memória do computador, que possui um nome (um rótulo) e pode guardar um valor por vez.

Contudo, o computador precisa saber que *tipo* de informação cada caixa pode armazenar. Não se pode guardar um texto em uma caixa destinada apenas a números inteiros. Essa característica é definida pelo **tipo de dado** da variável. Em Portugal, existem quatro tipos de dados primitivos (fundamentais):

1. **inteiro**: Usado para armazenar números inteiros, positivos ou negativos, sem casas decimais. Exemplos: 10, 0, -150.
2. **real**: Usado para armazenar números que podem ter casas decimais (também conhecidos como pontos flutuantes). Exemplos: 3.14, -0.5, 99.90.
3. **caractere**: Usado para armazenar sequências de texto, como nomes, frases ou endereços. O valor deve estar entre aspas duplas. Exemplos: "João da Silva", "Rua A, 123", "sim".
4. **logico**: Usado para armazenar valores booleanos, que só podem ser verdadeiro ou falso. É fundamental para a tomada de decisões no código.

Para usar uma variável, primeiro é preciso "declarar" sua existência, informando seu nome e seu tipo. A declaração de variáveis em Portugal é feita em uma seção especial, logo após a linha algoritmo e antes da palavra início, usando a palavra-chave var. A sintaxe é <nome_da_variavel>: <tipo_de_dado>.

Snippet de código

```
algoritmo "DeclaracaoVariaveis"
```

```
var
```

```
    nomeAluno: caractere
```

```
    idade: inteiro
```

```
    mediaFinal: real
```

```
    aprovado: logico
```

```
inicio
```

```
    // O programa começa aqui, mas por enquanto não faremos nada com as variáveis.
```

```
    // Apenas as declaramos para uso futuro.
```

```
fimalgoritmo
```

A escolha correta do tipo de dado é crucial para a eficiência e a correção do programa. Utilizar um tipo inteiro para o preço de um produto, por exemplo, impediria o armazenamento dos centavos, levando a cálculos incorretos.

Tipo de Dado	Descrição	Exemplo de Valor	Exemplo de Declaração
inteiro	Armazena números inteiros, sem casas decimais.	25, -100, 0	quantidade: inteiro
real	Armazena números com casas decimais.	1.75, 9.99, -273.15	altura: real
caractere	Armazena texto (letras, números,	"Maria", "CPF: 123.456"	nome: caractere

	símbolos).		
logico	Armazena apenas os valores verdadeiro ou falso.	verdadeiro, falso	matriculado: logico

2.4 Interação e Diálogo:

Lendo Dados do Usuário com `leia()`

Programas se tornam verdadeiramente úteis quando são interativos, ou seja, quando podem receber dados do usuário durante sua execução. O comando `escreva()` permite que o programa "fale" com o usuário; o comando `leia()` permite que o programa "ouça" o usuário.

O comando `leia()` pausa a execução do programa e espera que o usuário digite algo no console e pressione "Enter". O valor digitado é então armazenado na variável especificada dentro dos parênteses.

A boa prática de programação dita que, antes de um comando `leia()`, deve haver um `escreva()` para instruir o usuário sobre qual informação ele deve fornecer. Vamos combinar tudo o que aprendemos até agora em um programa completo e interativo:

Snippet de código

```
algoritmo "CadastroSimples"
```

```
// Este programa solicita o nome e a idade do usuário e depois exibe uma mensagem personalizada.
```

```
var
```

```
    nomeUsuario: caractere
```

```
    idadeUsuario: inteiro
```

```
inicio
```

```
    // Passo 1: Solicitar e ler o nome do usuário
```

```
    escreva("Por favor, digite o seu nome: ")
```

```
    leia(nomeUsuario)
```

```
    // Passo 2: Solicitar e ler a idade do usuário
```

```
    escreva("Agora, digite a sua idade: ")
```

```
    leia(idadeUsuario)
```

```
    // Passo 3: Exibir a mensagem final combinando os dados
```

```
    escreval("") // Pula uma linha para organizar a saída
```

```
    escreval("Olá, ", nomeUsuario, "! Você tem ", idadeUsuario, " anos.")
```

```
fimalgoritmo
```

Interação no Console:

```
Por favor, digite o seu nome: Djalma
```

```
Agora, digite a sua idade: 40
```

```
Olá, Djalma! Você tem 40 anos.
```

Este exemplo demonstra o fluxo fundamental de muitos programas:

1. **Entrada:** Coletar dados do usuário (leia).
2. **Processamento:** Neste caso, o processamento é simplesmente armazenar os dados.

3. **Saída:** Exibir uma resposta ou resultado com base nos dados de entrada (escreval).

2.5 Exercícios Práticos:

Entrada, Processamento e Saída

Para solidificar os conceitos deste capítulo, pratique com os seguintes exercícios:

1. **Soma Simples:** Crie um algoritmo que leia dois números inteiros e mostre na tela o resultado da soma deles. A saída deve ser formatada, por exemplo: "A soma entre 5 e 3 é 8".
2. **Cadastro de Produto:** Elabore um programa que leia o nome de um produto, seu preço (que pode ter centavos) e a quantidade em estoque. Ao final, exiba uma mensagem como: "Produto: Notebook, Preço: R\$ 3500.50, Estoque: 15 unidades".
3. **Conversor de Moedas Simples:** Escreva um programa que leia um valor em dólares (USD) e a cotação do dólar para o real (BRL). Calcule e exiba o valor correspondente em reais.

Capítulo 3:

Manipulando Dados: Operadores e Expressões

3.1 A Matemática do Computador:

Operadores Aritméticos

Uma vez que os dados estão armazenados em variáveis, o próximo passo é manipulá-los. A forma mais comum de manipulação é através de cálculos matemáticos. O Portugol oferece um conjunto de operadores aritméticos para realizar essas operações.

Os operadores básicos são:

- + : Adição
- - : Subtração
- * : Multiplicação
- / : Divisão

Além destes, existem dois operadores especiais muito úteis:

- % ou mod: Módulo (resto da divisão inteira). Este operador é extremamente útil. Por exemplo, $10 \% 3$ resulta em 1, porque 10 dividido por 3 dá 3 e sobra 1. Uma aplicação comum é verificar se um número é par ou ímpar: um número é par se o resto da sua divisão por 2 for 0 ($\text{número} \% 2 = 0$).
- ^: Potenciação (em algumas versões do Portugol). Exemplo: 5^2 resulta em 25.

Quando uma expressão contém múltiplos operadores, o computador segue uma ordem de precedência para resolver a conta, semelhante à regra que aprendemos na escola (PEMDAS/BODMAS). A multiplicação e a divisão são resolvidas antes da adição e da subtração. Por exemplo, na expressão

$5 + 2 * 3$, o resultado é 11 (primeiro $2 * 3$, depois $+ 5$), e não 21.

Para forçar uma ordem de cálculo diferente, utilizam-se os parênteses ().

Operações dentro de parênteses são sempre resolvidas primeiro. Na expressão $(5 + 2) * 3$, o resultado é 21.

Snippet de código

```
algoritmo "CalculoMedia"
```

```
var
```

```
    nota1, nota2, nota3, media: real
```

```
inicio
```

```
    escreva("Digite a primeira nota: ")
```

```
    leia(nota1)
```

```
    escreva("Digite a segunda nota: ")
```

```
    leia(nota2)
```

```
    escreva("Digite a terceira nota: ")
```

```
    leia(nota3)
```

```
    media <- (nota1 + nota2 + nota3) / 3 // Os parênteses garantem que a soma ocorra  
antes da divisão
```

```
    escreval("A média final do aluno é: ", media)
```

```
fimalgoritmo
```

3.2 Fazendo Comparações:

Operadores Relacionais

Além de calcular, os programas precisam constantemente comparar valores para tomar decisões. Os operadores relacionais são usados para este fim, e o resultado de qualquer comparação é sempre um valor lógico: verdadeiro ou falso.

Os seis operadores relacionais em Portugol são:

1. = : Igual a
2. <> : Diferente de
3. > : Maior que
4. < : Menor que
5. >= : Maior ou igual a
6. <=: Menor ou igual a

Esses operadores são a base para a construção de perguntas que o programa pode responder. Por exemplo:

- idade >= 18 (A idade é maior ou igual a 18?)
- salario < 1500 (O salário é menor que 1500?)
- senhaDigitada = "12345" (A senha digitada é igual a "12345"?)

É crucial entender que a expressão idade >= 18 não produz um número, mas sim o valor verdadeiro (se a variável idade for 18, 19, 20...) ou falso (se for 17, 16, 15...). Essa produção de valores lógicos é o que alimenta as estruturas de decisão que serão vistas no próximo capítulo.

Snippet de código

```
algoritmo "TesteRelacional"
```

```
var
```

```
idade: inteiro
```

```
maiorDeldade: logico
```

```
inicio
```

```
escreva ("Digite sua idade: ")
```

```
leia(idade)
```

```
maiorDeldade <- idade >= 18 // A variável 'maiorDeldade' armazena o resultado da  
comparação
```

```
escreval ("É maior de idade? ", maiorDeldade)
```

```
fimalgoritmo
```

3.3 A Lógica Booleana:

Operadores Lógicos (E, OU, NAO)

Frequentemente, uma decisão depende de mais de uma condição. Por exemplo, para obter uma carteira de motorista, uma pessoa precisa ser maior de 18 anos **E** ser aprovada no exame de direção. Para se aposentar por uma regra específica, uma pessoa pode precisar ter 65 anos de idade **OU** 35 anos de contribuição. Os operadores lógicos são usados para combinar essas múltiplas expressões relacionais.

Os três operadores lógicos principais são:

1. **E (AND):** Retorna verdadeiro somente se **ambas** as condições que ele conecta forem verdadeiras. Se uma delas for falsa, o resultado total é falso.
 - **Exemplo:** (media $\geq$ 7.0) E (frequencia $\geq$ 75) - O aluno só é aprovado se tiver média boa E frequência boa.
2. **OU (OR):** Retorna verdadeiro se **pelo menos uma** das condições que ele conecta for verdadeira. Ele só retorna falso se ambas as condições forem falsas.
 - **Exemplo:** (formaPagamento = "credito") OU (formaPagamento = "debito")
- A compra é válida se o pagamento for no crédito OU no débito.
3. **NAO (NOT):** É um operador unário que inverte o valor lógico de uma expressão. O que é verdadeiro se torna falso, e o que é falso se torna verdadeiro.
 - **Exemplo:** NAO (temDebitos) - Se temDebitos for falso, a expressão NAO (temDebitos) se torna verdadeira.

A "Tabela Verdade" é uma ferramenta visual que ajuda a entender o comportamento desses operadores:

A	B	A E B	A OU B
verdadeiro	verdadeiro	verdadeiro	verdadeiro
verdadeiro	falso	falso	verdadeiro
falso	verdadeiro	falso	verdadeiro
falso	falso	falso	falso

A combinação de operadores relacionais (que produzem valores lógicos) com

operadores lógicos (que combinam valores lógicos) permite a criação de expressões de decisão extremamente poderosas e complexas.

3.4 Exercícios Práticos:

Construindo Expressões Complexas

1. **Calculadora de IMC:** Crie um programa que leia o peso (em kg) e a altura (em metros) de uma pessoa. Calcule e exiba o Índice de Massa Corporal (IMC) usando a fórmula: $IMC = \text{peso} / (\text{altura} * \text{altura})$.
2. **Verificador de Aposentadoria:** Escreva um algoritmo que leia a idade e o tempo de contribuição de uma pessoa. O programa deve exibir verdadeiro se a pessoa pode se aposentar, e falso caso contrário. As regras para aposentadoria são: ter pelo menos 65 anos de idade **OU** ter pelo menos 30 anos de contribuição.
3. **Acesso ao Parque:** Um parque de diversões tem a seguinte regra para uma montanha-russa: a pessoa deve ter mais de 1.50m de altura **E** não pode ter problemas cardíacos. Crie um programa que leia a altura da pessoa e se ela tem problemas cardíacos (verdadeiro ou falso). O programa deve exibir na tela "Acesso liberado" ou "Acesso negado".

Capítulo 4: Tomando Decisões:

Estruturas Condicionais

4.1 O Caminho Duplo:

A Estrutura se-entao-senao

Programas se tornam inteligentes quando podem tomar decisões e alterar seu fluxo de execução com base em certas condições. A estrutura condicional mais fundamental e utilizada é o se-entao-senao (em inglês, if-then-else). Ela permite que o programa escolha entre diferentes caminhos.

A estrutura pode ser usada de duas formas:

1. Condicional Simples (se-entao)

Esta forma executa um bloco de código somente se a condição for verdadeira. Se a condição for falsa, o bloco é simplesmente ignorado e o programa continua sua execução normal.

Sintaxe:

Snippet de código

```
se <condicao_logica> entao
    // Bloco de código a ser executado se a condição for VERDADEIRA
fimse
```

Exemplo Prático: Um programa que dá um bônus a um vendedor apenas se ele atingir a meta.

Snippet de código

```
algoritmo "BonusVendedor"
var
    vendas, salario: real
inicio
    escreva("Digite o valor das vendas do mês: ")
    leia(vendas)
    escreva("Digite o salário base: ")
    leia(salario)

    se vendas > 10000 entao
        salario <- salario + 500
        escreval("Parabéns! Você atingiu a meta e recebeu um bônus de R$ 500,00.")
    fimse

    escreval("Seu salário final este mês é: R$ ", salario)
finalgoritmo
```

2. Condicional Composta (se-entao-senao)

Esta forma é mais completa, pois oferece dois caminhos de execução: um para o caso da condição ser verdadeira e outro para o caso de ser falsa. Isso garante que um dos dois blocos será sempre executado.

Sintaxe:

Snippet de código

```
se <condicao_logica> entao
    // Bloco de código para a condição VERDADEIRA
senao
```

```
// Bloco de código para a condição FALSA  
fimse
```

Exemplo Prático: Um programa que verifica se um aluno foi aprovado ou reprovado.

Snippet de código

```
algoritmo "VerificaAprovacao"  
var  
    media: real  
inicio  
    escreva("Digite a média final do aluno: ")  
    leia(media)  
  
    se media >= 7.0 entao  
        escreval("Aluno APROVADO!")  
    senao  
        escreval("Aluno REPROVADO.")  
    fimse  
fimalgoritmo
```

4.2 Múltiplas Possibilidades:

A Estrutura escolha-caso

Quando um programa precisa testar uma única variável contra vários valores possíveis, usar uma longa cadeia de se-senao se-senao se... pode tornar o código confuso e difícil de ler. Para essas situações, existe uma estrutura mais elegante e organizada: o escolha-caso (em inglês, switch-case).

Esta estrutura avalia uma variável e executa o bloco de código correspondente ao valor encontrado. Ela também oferece uma opção padrão (outro caso) para quando nenhum dos valores especificados corresponde.

Sintaxe:

Snippet de código

```
escolha <variavel>  
  caso <valor1>  
    // Bloco de código para o valor1  
  caso <valor2>  
    // Bloco de código para o valor2  
  //... outros casos  
  outrocaso  
    // Bloco de código padrão, se nenhum caso corresponder  
fimescolha
```

Exemplo Prático: Um programa que exibe o nome de um setor com base no código digitado.

Snippet de código

```
algoritmo "MenuSetores"  
var  
  codigoSetor: inteiro  
inicio  
  escreva("Digite o código do setor:")  
  escreva("1 - Financeiro")
```

```
escreval("2 - Recursos Humanos")
```

```
escreval("3 - Produção")
```

```
escreval("4 - Marketing")
```

```
leia(codigoSetor)
```

```
escolha codigoSetor
```

```
    caso 1
```

```
        escreval("Você selecionou o setor Financeiro.")
```

```
    caso 2
```

```
        escreval("Você selecionou o setor de Recursos Humanos.")
```

```
    caso 3
```

```
        escreval("Você selecionou o setor de Produção.")
```

```
    caso 4
```

```
        escreval("Você selecionou o setor de Marketing.")
```

```
    outrocaso
```

```
        escreval("Opção inválida! O código do setor não existe.")
```

```
fimescolha
```

```
fimalgoritmo
```

A decisão de usar um escolha-caso em vez de múltiplos se aninhados é uma questão de clareza e intenção. Se a lógica do programa depende de verificar o valor de uma única variável contra múltiplas constantes, o escolha-caso quase sempre resultará em um código mais limpo e legível.

4.3 Aninhando Decisões:

Condicionais Dentro de Condicionais

As estruturas condicionais podem ser "aninhadas", o que significa colocar uma estrutura se dentro de outra. Isso permite a criação de lógicas de decisão mais complexas e refinadas, onde uma segunda verificação só é realizada se a primeira for satisfeita.

Exemplo Prático:

Um sistema de e-commerce que oferece frete grátis para compras acima de R\$ 200,00, mas apenas para clientes da região Sudeste.

Snippet de código

```
algoritmo "CalculoFrete"
```

```
var
```

```
    valorCompra: real
```

```
    regioao: caractere
```

```
inicio
```

```
    escreva("Digite o valor total da compra: R$ ")
```

```
    leia(valorCompra)
```

```
    escreva("Digite a sigla da sua região (ex: SE, S, NE, N, CO): ")
```

```
    leia(regiao)
```

```
    se valorCompra > 200.0 entao
```

```
        // Primeira condição atendida: compra acima de 200
```

```
        se regioao = "SE" entao
```

```
            // Segunda condição (aninhada) atendida: cliente do Sudeste
```

```
            escreval("Parabéns! Você ganhou frete grátis.")
```

```
        senao
```

```
            // Segunda condição não atendida: cliente de outra região
```

```
    escreval("Sua compra se qualifica para um desconto no frete.")
fimse
senao
    // Primeira condição não atendida: compra abaixo de 200
    escreval("O frete para sua região será calculado normalmente.")
fimse
finalgoritmo
```

O aninhamento permite criar uma árvore de decisões, onde cada galho representa uma nova condição a ser verificada, tornando o programa capaz de lidar com cenários multifacetados.

4.4 Exercícios de Fixação:

Resolvendo Problemas com Condicionais

1. **Calculadora de IMC Completa:** Expanda o exercício do capítulo anterior. Após calcular o IMC, o programa deve exibir a classificação da pessoa de acordo com a tabela: Abaixo de 18.5 (Abaixo do peso), Entre 18.5 e 24.9 (Peso normal), Entre 25 e 29.9 (Sobrepeso), 30 ou mais (Obesidade). (Use se-senao se).
2. **Caixa Eletrônico:** Crie um programa que simule um menu de caixa eletrônico com as opções: 1-Ver Saldo, 2-Sacar, 3-Depositar. Use a estrutura escolha-caso para executar a ação escolhida pelo usuário.
3. **Verificador de Empréstimo:** Um banco concede empréstimos sob duas condições: o solicitante deve ter um salário mensal acima de R\$ 3000,00 e não pode ter restrições no CPF. Crie um programa que leia o salário e se a pessoa tem restrição (verdadeiro ou falso). O programa deve usar condicionais aninhadas para exibir "Empréstimo Aprovado" ou "Empréstimo Negado", informando o motivo da negação se for o caso.

4. **Divisibilidade:** Escreva um algoritmo que leia um número inteiro e informe se ele é divisível por 3 e por 7 ao mesmo tempo.

Capítulo 5:

Executando Tarefas Repetitivas: Laços de Repetição

5.1 Repetição Controlada:

O Laço para

Muitas tarefas em programação são inerentemente repetitivas, como processar todos os itens de uma lista, calcular a tabuada de um número ou exibir uma contagem. Em vez de escrever o mesmo código várias vezes, utilizamos **laços de repetição**.

O laço para (em inglês, for) é a estrutura ideal quando o número de repetições é conhecido de antemão. Ele utiliza uma variável de controle (um contador) que é inicializada com um valor, e o laço continua a ser executado até que essa variável atinja um valor final, sendo incrementada a cada iteração.

Sintaxe:

Snippet de código

```
para <variavel_contadora> de <valor_inicial> ate <valor_final> faca  
    // Bloco de código a ser repetido  
fimpara
```

Exemplo Prático: Um programa que calcula e exibe a tabuada de um número informado pelo usuário.

Snippet de código

```
algoritmo "Tabuada"
var
    numero, i, resultado: inteiro
inicio
    escreva("Digite um número para ver sua tabuada: ")
    leia(numero)
    escreval("--- Tabuada do ", numero, " ---")

    // O laço 'para' vai repetir o bloco de código 10 vezes.
    // A variável 'i' começará em 1 e irá até 10.
    para i de 1 ate 10 faca
        resultado <- numero * i
        escreval(numero, " x ", i, " = ", resultado)
    fimpara
finalgoritmo
```

Neste exemplo, o laço para automatiza a repetição, tornando o código conciso e eficiente. Sem ele, seria necessário escrever a linha escreval dez vezes, uma para cada multiplicação.

5.2 Repetição Condicional (Pré-Teste):

O Laço enquanto

E quando não sabemos de antemão quantas vezes uma tarefa precisa ser repetida? Por exemplo, em um jogo de adivinhação, o programa deve continuar pedindo um palpite até que o usuário acerte. Isso pode levar uma, cinco ou cinquenta tentativas.

Para esses casos, utilizamos o laço enquanto (em inglês, while). Este laço executa um bloco de código repetidamente **enquanto** uma condição lógica for verdadeira. A característica principal é que a condição é testada **antes** de cada execução do bloco. Se a condição for falsa desde o início, o bloco de código dentro do laço nunca será executado.

Sintaxe:

Snippet de código

```
enquanto <condicao_logica> faca  
    // Bloco de código a ser repetido  
fimenquanto
```

Exemplo Prático: Um programa que soma números digitados pelo usuário até que ele digite o número 0.

Snippet de código

```
algoritmo "SomaAteZero"  
var  
    numero, soma: inteiro  
inicio
```

```
soma <- 0
escreva("Digite um número para somar (ou 0 para sair): ")
leia(numero)

// O laço continua enquanto o número digitado for diferente de 0
enquanto numero <> 0 faça
    soma <- soma + numero
    escreval("Soma atual: ", soma)
    escreva("Digite outro número (ou 0 para sair): ")
    leia(numero)
fimenquanto
escreval("A soma final de todos os números foi: ", soma)
finalgoritmo
```

5.3 Repetição Condicional (Pós-Teste):

O Laço faça-enquanto (repita-ate)

Existe uma variação do laço condicional chamada faça-enquanto (ou repita-ate no VisuAlg). Sua principal diferença em relação ao enquanto é o momento em que a condição é verificada.

Neste laço, o bloco de código é executado primeiro, e a verificação da condição ocorre **no final**. Isso garante que o bloco de código será executado **pelo menos uma vez**, independentemente da condição.

Esta estrutura é ideal para situações como menus de opções, onde o menu precisa ser exibido ao menos uma vez antes de o usuário decidir se quer continuar ou sair.

É importante notar uma diferença de sintaxe e lógica entre as ferramentas:

- **VisuAlg (repita-ate):** O laço continua até que a condição se torne **verdadeira** (condição de parada).
- **Portugol Studio (faca-enquanto):** O laço continua enquanto a condição for **verdadeira** (condição de continuação).

Vamos focar na sintaxe do Portugol Studio, que é mais comum em linguagens modernas.

Sintaxe (Portugol Studio):

Snippet de código

```
faca {  
    // Bloco de código a ser repetido  
} enquanto (<condicao_logica_de_continuacao>)
```

Exemplo Prático: Um programa de menu que só encerra quando o usuário digita a opção de sair.

Snippet de código

```
algoritmo "MenuRepeticao"  
var  
    opcao: inteiro  
inicio  
    faca {  
        escreval("--- MENU ---")  
        escreval("1. Cadastrar item")  
        escreval("2. Listar itens")  
        escreval("3. Sair")  
        escreva("Escolha uma opção: ")
```

```
leia(opcao)

// Aqui entraria o código para cada opção (usando 'escolha-caso')

} enquanto (opcao <> 3) // O laço se repete enquanto a opção for diferente de 3
escreval("Programa encerrado.")

fimalgoritmo
```

A escolha entre enquanto e faça-enquanto depende da lógica do problema: se a ação pode ou não acontecer (pré-teste), use enquanto. Se a ação deve acontecer pelo menos uma vez (pós-teste), use faça-enquanto.

5.4 Exercícios de Aplicação:

Dominando os Laços de Repetição

1. **Contagem Regressiva (para):** Crie um programa que peça um número ao usuário e, em seguida, mostre uma contagem regressiva a partir desse número até 0.
2. **Validação de Nota (enquanto):** Escreva um programa que peça ao usuário para digitar uma nota entre 0 e 10. O programa deve continuar pedindo a nota **enquanto** o valor digitado for inválido (menor que 0 ou maior que 10).
3. **Média da Turma (faça-enquanto):** Elabore um algoritmo que leia as idades de vários alunos. O programa deve parar de ler idades quando uma idade negativa for digitada. No final, deve exibir a média de idade da turma. O uso do faça-enquanto ou repita-ate é adequado aqui, pois pelo menos uma idade precisa ser lida.
4. **Tabuadas de 1 a 10 (para aninhado):** Crie um programa que mostre na tela as tabuadas de todos os números de 1 a 10. Você precisará de um laço para dentro de outro.

Estrutura	Ideal para...	Teste da Condição	Garante Execução?
para	Repetições com contagem definida e conhecida.	No início (implícito no contador).	Depende do contador (se início > fim, não executa).
enquanto	Repetições que podem não ocorrer nenhuma vez.	No início (Pré-teste).	Não. Se a condição for falsa de início, não executa.
faca-enquanto	Repetições que devem ocorrer ao menos uma vez.	No final (Pós-teste).	Sim, ao menos uma vez.

Apêndice A:

Gabarito Comentado de Exercícios Selecionados

Esta seção fornece soluções comentadas para alguns dos exercícios propostos, com foco em explicar não apenas "o quê" o código faz, mas "por que" uma determinada estrutura foi escolhida.

Exercício 4.1:

Calculadora de IMC Completa

Snippet de código

```
algoritmo "CalculoIMCCompleto"
```

```
// Objetivo: Calcular o IMC e classificar o resultado usando condicionais encadeadas.
```

```
// Estrutura escolhida: se-senao se, pois há múltiplas faixas de valores
```

```
// que não são valores únicos, tornando o 'escolha-caso' inadequado.
```

```
var
```

```
    peso, altura, imc: real
```

```
inicio
```

```
    escreva("Digite seu peso em kg (ex: 70.5): ")
```

```
    leia(peso)
```

```
    escreva("Digite sua altura em metros (ex: 1.75): ")
```

```
    leia(altura)
```

```
    // Cálculo do IMC
```

```
    imc <- peso / (altura * altura)
```

```
    escreval("Seu IMC é: ", imc)
```

```
    // Estrutura condicional para classificar o IMC
```

```
    se imc < 18.5 entao
```

```
        escreval("Classificação: Abaixo do peso.")
```

```
    senao se imc < 25 entao // Se não for < 18.5, já sabemos que é >= 18.5
```

```
        escreval("Classificação: Peso normal.")
```

```
    senao se imc < 30 entao // Se não for < 25, já sabemos que é >= 25
```

```
        escreval("Classificação: Sobrepeso.")
```

```
    senao // Se não se encaixou em nenhuma das anteriores, só pode ser >= 30
```

```
        escreval("Classificação: Obesidade.")
```

```
    fimse
```

fimalgoritmo

Exercício 5.2: Validação de Nota

Snippet de código

algoritmo "ValidacaoDeNota"

// Objetivo: Forçar o usuário a digitar um valor dentro de um intervalo válido.

// Estrutura escolhida: 'enquanto', pois a validação deve ocorrer ANTES

// de o programa prosseguir. Se o usuário digitar a nota correta de primeira,

// o laço nem precisa ser executado.

var

nota: real

inicio

escreva("Digite a nota do aluno (entre 0 e 10): ")

leia(nota)

// O laço de repetição continua ENQUANTO a nota for inválida.

// A condição para ser inválida é: (nota < 0) OU (nota > 10).

enquanto (nota < 0) ou (nota > 10) faca

escreval("Valor inválido!")

escreva("Por favor, digite uma nota entre 0 e 10: ")

leia(nota)

fimenquanto

// Quando o programa sai do laço, temos a garantia de que a nota é válida.

escreval("A nota digitada foi ", nota, " e é válida.")

fimalgoritmo

Exercício 5.4:

Tabuadas de 1 a 10 (Laços Aninhados)

Snippet de código

```
algoritmo "TabuadasCompletas"
```

```
// Objetivo: Exibir a tabuada de todos os números de 1 a 10.
```

```
// Estrutura escolhida: 'para' aninhado. O laço externo controla qual
```

```
// tabuada está sendo calculada (a do 1, a do 2, etc.). O laço interno
```

```
// calcula cada linha dessa tabuada (1x1, 1x2, etc.).
```

```
var
```

```
  i, j, resultado: inteiro
```

```
inicio
```

```
  // Laço externo: controla o número da tabuada (de 1 a 10)
```

```
  para i de 1 ate 10 faca
```

```
    escreval("-----")
```

```
    escreval("--- Tabuada do ", i, " ---")
```

```
    escreval("-----")
```

```
    // Laço interno: controla o multiplicador (de 1 a 10)
```

```
    para j de 1 ate 10 faca
```

```
      resultado <- i * j
```

```
      escreval(i, " x ", j, " = ", resultado)
```

```
    fimpara
```

```
    escreval("") // Pula uma linha para separar as tabuadas
```

```
  fimpara
```

```
fimalgoritmo
```